

Predictive Modeling of Horse Racing Market Dynamics on Betfair Exchange Using CNN-LSTM and XGBoost Models

Candidate Number: NZTB1

Supervisor : Dr Lo, Chak Hei

Word Count: 9, 120

Department of Statistical Science
University College London

September 1, 2025

Abstract

This study aims to advance predictive modeling in betting exchanges by bridging the strengths of deep sequence modeling and ensemble-based approaches. Convolutional neural networks (CNN) and long short-term memory (LSTM) networks are employed to capture localized patterns and temporal dependencies in sequential odds and volume data, hoping to identify hidden pattern that increase the model prediction power. XGBoost is employed to model static covariates and provide interpretability, while CNN-LSTM networks are trained on engineered tabular features derived from selected variables and optimized using a validation set. Furthermore, the resulting hybrid system is evaluated on a dataset comprising 1701 horse races conducted on the Betfair Exchange platform. The results demonstrate that the CNN model consistently outperforms a tree-based ensemble approach in both the odds movement and arbitrage detection tasks. Moreover, the combination of the two models increase the predictive power in some tasks. Also, the XGBoost model provided some interpretability to CNN-LSTM models by showing feature importance.

Acknowledgments

I would like to express my sincere gratitude to Smartodds for their invaluable support, guidance, and for providing access to the data that made this research possible. I am especially grateful to my supervisors and colleagues for their insightful feedback, encouragement, and expertise throughout the development of this project. Finally, I would like to thank my friends and family for their support and understanding during the course of this study.

Contents

1	Introduction	5
2	Literature Review	6
2.1	Problem Formulation	6
2.2	Existing Approaches	6
2.2.1	Traditional Methods and Early Probability Models	6
2.2.2	Time-Series Forecasting: ARIMA and Related Models	7
2.2.3	Machine Learning Approaches	7
2.2.4	Deep Learning Models	8
2.3	Theoretical Foundation of the Chosen Methods	8
2.4	CNN-LSTM for Sequential Modeling	8
2.4.1	Convolutional feature extraction	8
2.4.2	Recurrent temporal integration (LSTM)	9
2.4.3	Saliency Map	9
2.5	Extreme Gradient Boosting (XGBoost)	10
3	Data and Methodology	11
3.1	Data Preparation	11
3.1.1	Overview of Data Source	11
3.1.2	Feature Engineering	12
3.1.3	Data Representation and Partitioning	13
3.2	Prediction Task	14
3.2.1	Short-Term Price Movement Prediction	14
3.2.2	Riskless Arbitrage Opportunity Detection	15
3.2.3	Race Outcome Prediction	15
3.3	Model Architectures	15
3.4	CNN-LSTM Hybrid Network	16

3.4.1	Convolutional Feature Extraction	16
3.4.2	Sequential Modeling with LSTMs	16
3.4.3	Dense Layers and Nonlinear Mapping	17
3.4.4	Training Objective and Threshold Calibration	17
3.5	XGBoost Implementation	18
3.5.1	Baseline	18
3.5.2	Meta Model	18
3.6	Model Training Protocol	19
3.7	Evaluation Framework	19
4	Result and Discussion	20
4.1	Descriptive Statistics of the Dataset	20
4.2	Model Performance on Classification Tasks	21
4.3	Profitability Simulation Results	23
4.4	Saliency Map	26
4.5	Feature Importance	28
4.6	Comparison Between CNN-LSTM and XGBoost	29
4.7	Limitations	30
4.8	Practical Implications for Business Application	30
5	Conclusion	31
	References	32
	Appendices	34
A	Code Implementation	34
B	Arbitrage Derivation	34
B.0.1	Capital Allocation	34
B.0.2	Profit Conditions	34

B.0.3	Arbitrage Solution	34
B.0.4	Positivity Conditions	35
C	Use of GenAI	35

1 Introduction

Horse racing is a globally popular sport and one of the most established domains for sports betting. In the United Kingdom alone, there are approximately eight to ten horse races held daily, with each race consisting eight or more competing horses. These races generate substantial financial activity in betting markets, where both individual bettors and institutional traders actively trade odds based on real-time information. Betting systems are broadly divided into traditional bookmaker-dominated models and decentralized peer-to-peer exchanges such as Betfair. In these exchanges, prices fluctuate rapidly in response to collective beliefs and market activity. Unlike fixed-odds betting, where bookmakers set relatively static prices, exchange-based betting introduces high-frequency price adjustments driven by information flows, trader behavior, and hidden patterns that may reflect underlying horse performance. This dynamic market structure creates a challenging yet compelling environment for prediction, arbitrage detection, and informed decision making.

Predicting horse race outcomes and odds trajectories has attracted considerable research interest across fields such as econometrics, probability theory, machine learning, and financial modeling. Anticipating short-term odds movements has clear practical implications for traders seeking profitable strategies, while accurate outcome prediction provides value to both bettors and analysts. However, the predictive task is nontrivial, market odds aggregate most of the public and private information, making them difficult to outperform. Furthermore, betting exchanges exhibit nonlinear price dynamics, volatility clustering, and rapid market shifts that closely resemble high-frequency financial markets, thereby limiting the effectiveness of traditional linear forecasting techniques.

This study addresses these challenges by introducing a hybrid modeling framework that combines deep learning and ensemble-based approaches. Convolutional neural networks (CNN) and long short-term memory (LSTM) networks are employed to capture localized patterns and temporal dependencies in sequential odds and volume data, hoping to identify hidden pattern that increase the model prediction power. In parallel, Extreme Gradient Boosting (XGBoost) leverages static race-level and market features to model complex interactions in a more interpretable way. The study aims to compare the performance of the two models. And ultimately, by integrating these complementary techniques, the study aims to advance predictive modeling in betting exchanges by bridging the strengths of deep sequence learning with tree-based ensemble methods.

2 Literature Review

2.1 Problem Formulation

Horse racing outcomes and betting odds are influenced by a wide range of factors, including track conditions, horse fitness, jockey skill, and race characteristics. However, this paper focuses on predicting both the odds and race outcomes by analyzing the time series of betting odds and corresponding traded volumes. Unlike traditional bookmaker-driven betting markets, where odds are set by bookmakers and updated infrequently, the Betfair betting exchange operates as a peer-to-peer market, where odds fluctuate continuously based on the aggregated beliefs of thousands of participants. This structure shares similarities with financial markets, making modeling approaches from finance highly relevant to betting exchange prediction problems (Goodacre and Schlagman, 2024).

Betting exchanges like Betfair provide a dynamic modeling environment where odds reflect real-time market trend, and prices adjust rapidly as new information enters the market. The central focus of this research are predicting odds movement and race outcomes as classification problems assigning probabilities to each horse at each time point. Prior studies such as (Øvregård, 2008) explored in-play odds prediction in tennis, while others applied time-series analysis and machine learning models to horse racing outcome prediction (Bunyan, 2014; Santos, 2014). The significance of this research lies in the unique opportunities presented by exchange-based betting markets, where profit-making strategies such as back-to-lay trading are possible, and market inefficiencies may create opportunities for arbitrage (Goodacre and Schlagman, 2024).

Forecasting betting odds is challenging because markets are noisy and highly volatile, making traditional techniques such as moving averages insufficient. Similarly, predicting race outcomes is difficult because market odds already aggregate public and private information, including horse condition, jockey skill, and weather, often reflecting highly accurate implied probabilities. Beating the market therefore requires identifying subtle temporal patterns or inefficiencies that may arise from the presence of informed traders or insider activity, as suggested in (Shin, 1991). Understanding these dynamics makes the study of odds time series an essential foundation for building predictive models of both price movements and race results.

2.2 Existing Approaches

2.2.1 Traditional Methods and Early Probability Models

Early studies on horse race prediction relied heavily on statistical modeling and econometric techniques. Logistic regression and related probabilistic models (Harville, 1973) became standard tools for predicting race outcomes due to their interpretability and probabilistic nature. These models estimated winning probabilities based empirical distribution of odds often strug-

gled with calibration, commonly overestimating probabilities for horses finishing in second or third place.

Extensions of logistic regression incorporated ranking models based on assumed probability distributions of finishing positions. For example, (Bacon-Shone et al., 1992) combined logistic regression with ranking approaches, while (Ali, 1998) tested these method with normal and gamma ranking models. Although these approaches provided a stronger theoretical basis, they consistently displayed systematic biases, particularly in estimating probabilities for non-winning horses.

2.2.2 Time-Series Forecasting: ARIMA and Related Models

Since betting odds evolve over time, researchers explored time-series forecasting methods, particularly the AutoRegressive Integrated Moving Average (ARIMA) model. ARIMA, widely used in finance and econometrics, assumes stationarity and linearity in time-series data, making it a natural choice for modeling sequential market data. However, betting exchanges are highly nonlinear systems driven by heterogeneous agents, and ARIMA’s assumptions often fail in such contexts.

Studies such as (Bunyan, 2014) applied ARIMA and related trend regression techniques to horse race forecasting, concluding that time-series methods performed poorly in predicting odds dynamics. Betting markets tend to react sharply to new information, leading to discontinuous patterns that linear models cannot effectively capture. Consequently, traditional time-series approaches have seen limited success in both odds and outcome prediction tasks.

2.2.3 Machine Learning Approaches

The rise of machine learning introduced more flexible, nonlinear models capable of handling complex interactions. Methods such as support vector machines (SVM), perceptrons, and decision trees emerged as promising alternatives to regression-based methods. Decision trees, in particular, attracted attention for their interpretability and ability to handle nonlinear relationships.

Boosting algorithms like XGBoost have shown strong performance in financial prediction tasks and have been applied to betting markets. (Terawong and Cliff, 2024) demonstrated that XGBoost could adaptively learn profitable in-play betting strategies and achieve competitive performance in predicting race outcomes. Unlike simpler models, boosting methods iteratively refine predictions by focusing on misclassified instances, making them well-suited for noisy markets such as Betfair.

2.2.4 Deep Learning Models

To overcome the limitations of traditional machine learning, researchers have explored deep learning methods, particularly artificial neural networks (ANNs). Early ANN studies used feedforward networks to model non-linear relationships between race features, achieving higher predictive accuracy than logistic regression (Dzalbs and Kalganova, 2018).

More recently, advanced architectures such as convolutional neural networks (CNN) and long short-term memory networks (LSTMs) have been applied to model sequential and temporal betting data (Gonçalves et al., 2019). CNN excel at detecting localized patterns in time-series data, while LSTMs are specifically designed to capture long-term dependencies, making them particularly effective in sequential financial or betting market prediction. Studies have shown that these deep models outperform traditional methods in odds movement forecasting, although they come with challenges such as long training times, computational costs, and limited interpretability. Neural networks often function as “black-box” models, making it difficult to understand how individual features influence predictions, which is a drawback for decision-making in betting environments.

2.3 Theoretical Foundation of the Chosen Methods

Based on the results of related works, this research applies a dual modeling strategy that integrates deep learning and gradient boosting to leverage the strengths of both approaches. The Convolutional Long Short-Term Memory (CNN-LSTM) network is used to capture sequential dependencies in betting odds, while XGBoost is employed to model static covariates and provide interpretability.

2.4 CNN-LSTM for Sequential Modeling

Let $\{\mathbf{x}_t\}_{t=1}^T$ denote a multivariate time series of length T , where each $\mathbf{x}_t \in \mathbb{R}^F$ collects F market microstructure features at time t (e.g., back/lay odds, traded volumes). The goal is to learn a mapping from the observed sequence $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_T) \in \mathbb{R}^{T \times F}$ to a target $\hat{y} \in [0, 1]$ to approximate $P(y | \mathbf{X})$ in classification, where $y \in \{0, 1\}$ represent the binary classification (Lu et al., 2020).

2.4.1 Convolutional feature extraction

A 1D convolutional layer aims to transform local windows of the input into higher-level features by applying N filters to the input matrix $\mathbf{X}$. For the n -th filter at time point t , the activation step is

$$z_t^{(n)} = \text{ReLU} \left(\sum_{i=0}^K \sum_{j=0}^F W_{i,j}^{(n)} \tilde{X}_{t+i,j} + b^{(n)} \right)$$

, where $\text{ReLU}(x) = \max\{0, x\}$ is the activation function, $W_{i,j}^{(n)}$ is the weight at position (i, j) for the n -th filter, $\tilde{\mathbf{X}} \in \mathbb{R}^{(T+K-2) \times F}$ is the zero-padded input of $\mathbf{X}$, and K is the size of the kernel.

Stacking all N filters yields a convolved feature vector $\mathbf{z}_t = (z_t^{(1)}, \dots, z_t^{(N)})^\top \in \mathbb{R}^N$ at each t . Optional temporal downsampling via max/average pooling with pool size P reduces the sequence length from T to $\tilde{T} = \lfloor \frac{T}{P} \rfloor$, trading temporal resolution for robustness to local noise and small time warps in odds streams. Multiple convolutional blocks can be stacked, optionally with batch normalization and dropout.

2.4.2 Recurrent temporal integration (LSTM)

The sequence of convolved features $\{\mathbf{z}_t\}_{t=1}^{\tilde{T}}$ is passed to a Long Short-Term Memory (LSTM) layer to model longer-range dependencies. At each time step t , the LSTM updates its hidden state $\mathbf{h}_t \in \mathbb{R}^H$ and cell state $\mathbf{c}_t \in \mathbb{R}^H$ via gated recurrences:

$$\begin{aligned} \mathbf{f}_t &= \sigma([\mathbf{h}_{t-1}, \mathbf{z}_t] \mathbf{W}_f) \\ \mathbf{i}_t &= \sigma([\mathbf{h}_{t-1}, \mathbf{z}_t] \mathbf{W}_i) \\ \mathbf{o}_t &= \sigma([\mathbf{h}_{t-1}, \mathbf{z}_t] \mathbf{W}_o) \\ \tilde{\mathbf{c}}_t &= \tanh([\mathbf{h}_{t-1}, \mathbf{z}_t] \mathbf{W}_c), \\ \mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \\ \mathbf{h}_t &= \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \end{aligned}$$

where $\mathbf{f}_t$, $\mathbf{i}_t$ and $\mathbf{o}_t$ are the forget gate, input gate and output gate respectively, $\sigma(\cdot)$ is the logistic sigmoid, $\odot$ denotes elementwise multiplication, and $\mathbf{W}_{(\cdot)}$ are trainable parameters. The forget, input, and output gates $(\mathbf{f}_t, \mathbf{i}_t, \mathbf{o}_t)$ regulate information flow, enabling the model to retain or discard past evidence about market pressure encoded in $\mathbf{c}_t$.

For sequence-to-one prediction, the final hidden state $\mathbf{h}_{\tilde{T}}$ summarizes the sequence. The prediction head depends on the task is

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{h}_{\tilde{T}} + b)$$

2.4.3 Saliency Map

Although deep learning models are often regarded as black-box systems, techniques such as saliency maps have been developed to provide interpretability by visualizing how the model makes predictions. A saliency map identifies the parts of the input that contribute most

strongly to the output by computing the gradient of the model’s prediction score with respect to each input feature (Alqaraawi et al., 2020).

Formally, for a model output S_c representing the score for class c , and an input $x \in \mathbb{R}^n$, the saliency of each feature x_i is computed as:

$$M_i = \left| \frac{\partial S_c}{\partial x_i} \right|,$$

where M_i measures how sensitive the prediction is to changes in the input feature x_i . Large gradient values indicate that small perturbations in that input dimension would lead to significant changes in the output, implying that this feature is highly important for the model’s decision.

In practice, these gradients are often visualized as heatmaps over the input space, with brighter or warmer colors corresponding to higher saliency. In image classification tasks, this highlights the pixels that most strongly influence the model’s decision. In time-series modeling, as in this work, saliency maps highlight the time steps or variables that are most influential for predicting odds movement or race outcomes.

Saliency maps enable domain experts to interpret deep learning predictions by showing which parts of the input drive the model’s outputs. However, they have limitations, they are sensitive to noise, gradients can be locally unstable, and they provide only a first-order approximation of feature importance rather than capturing more complex interactions. Despite these challenges, saliency maps remain a powerful tool for diagnosing model behavior, identifying spurious correlations, and improving trust in model predictions (Alqaraawi et al., 2020).

2.5 Extreme Gradient Boosting (XGBoost)

While deep learning excels at extracting temporal features from sequential data, betting datasets also contain rich tabular information from race odds statistics, and market liquidity measures. Gradient boosting, and specifically the XGBoost algorithm (Chen and Guestrin, 2016), is well-suited for modeling this type of data. XGBoost builds an ensemble of decision trees sequentially, minimizing a regularized objective function that balances model complexity with predictive accuracy. At each boosting iteration, XGBoost fits a new decision tree to reduce the loss function. And by using only partial samples to build the tree, this makes XGBoost robust to overfitting while maintaining high predictive accuracy on structured data.

A significant advantage of XGBoost is its interpretability. One powerful interpretability method is SHAP (SHapley Additive exPlanations) (Lundberg and Lee, 2017), which assigns each feature a contribution value for every prediction, based on principles from cooperative game theory. The SHAP framework computes feature attributions by treating each feature as a contributor to the prediction and the ”payout” is the model prediction. The SHAP value for a feature i

is the average marginal contribution of that feature over all possible feature subsets (Ichiishi, 1983):

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N| - |S| - 1)!}{|N|!} [f_{S \cup \{i\}}(x_{S \cup \{i\}}) - f_S(x_S)],$$

,where N is the set of all features, S is a subset of features excluding i , $f_{S \cup \{i\}}$ is the model prediction when feature i is included, and f_S is the prediction without feature i .

In practice, exact computation of SHAP values is expensive because it involves evaluating all feature combinations (2^N subsets). For tree-based models like XGBoost, specialized algorithms compute SHAP values in polynomial time, making it feasible for large-scale datasets.

The combination of XGBoost with CNN-LSTM allows the system to leverage the temporal modeling power of deep learning while retaining interpretability through SHAP analysis. This hybrid modeling framework provides not only predictive power but also actionable insights, helping analysts understand which race characteristics or market factors influence odds dynamics and race outcomes.

3 Data and Methodology

3.1 Data Preparation

3.1.1 Overview of Data Source

The data set used in this study was obtained from the Betfair Exchange. It is a peer-to-peer betting platform that facilitates betting between participants without the intermediation of a traditional bookmaker. Unlike conventional betting systems, where the bookmaker sets fixed odds, the Exchange operates on a market-making model, enabling users to act as either backers (betting for an outcome) or layers (betting against an outcome). The odds in this betting system are dynamically determined by supply and demand, with participants able to submit orders or match existing orders at prevailing market rates, similar to financial markets such as stock exchanges.

Within this framework, back odds denote the price at which a bettor wagers on a given horse to win, whereas lay odds represent the price at which a bettor bets against the given horse winning. These odds exhibit continuous fluctuations, often experiencing rapid changes in the final minutes before the race, as well as during the race itself.

The data set comprises timestamped records of back/lay odds and corresponding order volumes for 14,195 individual horses across multiple races over a three-month period, capturing market snapshots at one-minute intervals from ten minutes before race commencement until the official start.

3.1.2 Feature Engineering

The predictive accuracy of machine learning models in highly dynamic betting markets critically depends on appropriate feature selection. This study systematically evaluates both raw market data and engineered features to identify those with the greatest predictive power. These features represent the market state for individual horses at each time point, providing the models with information on immediate market conditions and underlying structural imbalances.

At each timestamp, the dataset records eight fundamental market variables, providing full depth visibility to the two best price levels on both the back and lay sides of the order book. Specifically, the back side is characterised by the best back price ($b^{(0)}$) and corresponding volume ($V^{(b_0)}$) as well as the second-best back price ($b^{(1)}$) and volume ($V^{(b_1)}$). The lay side similarly has best lay price ($l^{(0)}$) and volume ($V^{(l_0)}$), and second-best lay price ($l^{(1)}$) and volume ($V^{(l_1)}$).

Using raw price and volume time series directly can provide meaningful insights. However, high correlation between each time point limit their predictive utility when used alone. To extract more robust and useful information, engineered time series features were created. These features are based on established measures from financial market analysis and sports betting research.

The Volume-Weighted Average Price (VWAP) was employed to produce the average price of the betting odds weighted by volume. It is commonly used in financial markets to assess whether trades are executed at favorable prices relative to overall market activity. In the context of Betfair's betting exchange, VWAP provides a smoothed and more robust representation of price movement than instantaneous best odds that accounts for liquidity effects:

$$VWAP = \frac{b_0 V_{b_0} + b_1 V_{b_1}}{V_{b_0} + V_{b_1}} \quad (3.1)$$

Price Volume Trend (PVT) is a indicator that combines price movements with trading volume to measure the strength and direction of market trends. It is particularly useful for detecting whether price changes are supported by significant trading activity:

$$PVT_t = PVT_{t-1} + \frac{VWAP_t - VWAP_{t-1}}{VWAP_{t-1}} \times V_t \quad (3.2)$$

where V_t is the total volume at time t . For each horse, its individual PVT is computed, as well as the aggregated PVT of all opponents in the same race, providing context on relative market trend.

To stabilise variance and enable additive modeling of price changes, the logarithmic returns of VWAP were computed. This transformation is standard in time series modeling of financial instruments and is here adapted to the betting odds domain:

$$r_t = \ln \left(\frac{VWAP_t}{VWAP_{t-1}} \right) \quad (3.3)$$

Weight of Money (WoM) was constructed to quantify imbalance across the back and lay sides

of the market, following formula in (Dzalbs and Kalganova, 2018):

$$WoM = \frac{2V_{b_0} + V_{b_1}}{2V_{b_0} + V_{b_1} + 2V_{l_0} + V_{l_1}} \quad (3.4)$$

This formulation doubles the weight of the volume available at the best price level, reflecting its greater market relevance. A WoM value exceeding 0.5 indicates more money backing than laying the horse, suggesting downward price pressure.

Finally, the implied probabilities of winning and losing are derived directly from VWAP at each time point. For each horse, the probability of winning is computed as the inverse of the back odds, while the probability of losing is calculated as one minus the inverse of the lay odds. These probabilities are then normalized to sum to 1 (Goodacre and Schlagman, 2024) within each race at every time point, ensuring comparability across horses and races regardless of market size.

In addition to sequential feature engineering, several statistical transformations were applied to the VWAP series to capture different aspects of its behavior. These include the maximum, the minimum, the mean, and the slope within a given window. The mean indicates the central tendency odds level, and the slope of the fitted linear trend measures directional momentum over the window. These transformations provide complementary perspectives on VWAP dynamics, allowing models such as XGBoost to leverage summary statistics that condense temporal patterns into informative tabular features.

Together, these engineered features aim to capture both the immediate market environment and latent price pressures, providing the predictive models with a more interpretable representation of market dynamics than raw prices and volumes alone.

3.1.3 Data Representation and Partitioning

The market dynamics for each horse are captured through a discrete time series of 11 observations, with the race commencement time serving as the final data point. Conversion of raw odds to log returns produces an effective series length of 10 temporal observations per horse. This compressed representation differs from conventional financial time series analysis, where models typically operate on substantially longer historical sequences, often comprising hundreds or thousands of observations.

The constrained temporal window introduces significant analytical limitations. With a restricted historical context, the model is unable to capture extended temporal dependencies, which often play a key role in improving forecast accuracy in financial modeling. Furthermore, sparse temporal sampling (60 seconds) requires each observation to have greater informational significance, as critical market movements may occur between measurement intervals.

To address these limitations, a sliding window approach (window size = 5, stride = 1) was used to generate overlapping temporal segments that augment the training data into 70975 samples.

The model is exposed to slightly shifted views of the same temporal patterns, which helps it better capture short-term dependencies within the limited time points. Moreover, strategic feature engineering at each time point enhanced the predictive signals within the time series. With each observation becomes more informative, partially compensating for the lack of extended historical context and sparse temporal sampling.

For model assessment, a race-level partition method was used to divide the dataset into training (40%), validation (10%), and test (50%) subsets chronologically, with horses in the same race are grouped in the same subset. This partitioning strategy ensured no horses from the same race appeared in multiple subsets, preventing data leakage and providing a realistic evaluation of generalisation performance. The validation set was used for hyperparameter optimisation, while the test set served as the definitive benchmark for performance metrics and profitability simulations.

3.2 Prediction Task

This study examines three prediction tasks within the Betfair Exchange market, each designed to capture a distinct aspect of market behavior and efficiency. Progressing from short-term price forecasting to the identification of arbitrage opportunities and ultimately to race outcome prediction, the tasks collectively provide a comprehensive framework for assessing the predictive power of machine learning in betting exchange environments.

3.2.1 Short-Term Price Movement Prediction

The first task concerns the prediction of directional movements in back odds over a 60-second horizon. This problem can be framed as a binary classification challenge, requiring the model to anticipate whether odds will increase (indicating declining market confidence in a horse) or decrease (reflecting growing demand to back the horse).

This formulation parallels established methodologies in financial market research, where the focus is on anticipating immediate price trend rather than long-run valuations. The key difficulty lies in distinguishing persistent trends from short-lived fluctuations, particularly in the presence of market noise and the non-stationary dynamics that intensify as race start time approaches.

Accurate prediction of odds movements enables bettors to optimize their entry points. When a bettor is confident that a horse is likely to win, the objective is to secure the highest available back odds, thereby maximizing potential returns if the prediction proves correct. Conversely, when the expectation is that a horse will lose, entering the market at the most favorable lay odds enables the bettor to minimize liability while still capitalizing on their informational advantage. In both cases, the ability to anticipate short-term fluctuations in odds supports more informed entry decisions, aligning bet placement with the most advantageous price levels.

3.2.2 Riskless Arbitrage Opportunity Detection

The second task examines the potential existence of arbitrage opportunities that allow for theoretically risk-free profits through sequential trading strategies. Such opportunities occur when predictable price paths enable traders to first back at higher odds and subsequently lay at lower odds, or conversely to lay first and then back at better odds, thereby guaranteeing a profit regardless of the race outcome.

The formalization (Appendix B) of these opportunities relies on the condition:

$$b > l \quad , \text{ where back odds is } b, \text{ lay odds is } l \text{ and regardless of commission rate} \quad (3.5)$$

In practice, a back-lay sequence assumes that a bettor places a back bet at time t and subsequently executes a lay bet at a later point if the arbitrage condition is satisfied. If this sequence produces a positive profit, it is classified as 1; otherwise, it is classified as 0. The reverse logic applies to lay-back sequences, where an initial lay bet is followed by a back bet under the conditions.

This formulation constitutes a binary classification problem, in which the model must predict whether the odds trend within a given time window will generate an actionable arbitrage opportunity. Unlike the short-term price movement prediction task, this problem requires capturing and forecasting longer-horizon odds movement, since arbitrage opportunities depend on sustained and directional shifts rather than momentary fluctuations.

3.2.3 Race Outcome Prediction

The third task focuses on race outcome prediction, formulated as a binary classification problem in which the target variable indicates whether a given horse wins or loses the race. The analysis investigates whether machine learning models can extract meaningful information from the temporal patterns and fluctuations of historical odds beyond what is captured by the latest market odds, thereby identifying potential sources of predictive advantage in outcome forecasting.

3.3 Model Architectures

Two distinct machine learning approaches were employed in this study. The first is a deep sequential model combining Convolutional Neural Networks (CNN) with Long Short-Term Memory (LSTM) model. This hybrid architecture is designed to capture both spatial correlations among features and temporal dependencies within the sequential betting data. The convolutional layers act as local pattern detectors, extract local temporal and cross-feature patterns. These representations are then passed to the LSTM layers, which preserve and exploit longer-range temporal dependencies, enabling the model to recognize how local fluctuations accumulate into broader trajectories. By integrating localized feature extraction with sequential

memory, the CNN–LSTM framework provides a flexible mechanism for modeling the evolving dynamics of betting exchange markets, where short-lived noise and sustained directional trends jointly determine market behavior.

The second approach uses tree-based gradient boosting through XGBoost and is designed to establish a strong baseline model to provide a benchmark for comparison with the CNN-LSTM model, which takes advantage of the full sequential structure of the data. Unlike sequence-based models, XGBoost does not explicitly account for temporal ordering; instead, it models non-linear interactions among features represented in a tabular format over the historical windows. This makes XGBoost a competitive reference model and allows for a evaluation of whether deep sequential architectures offer meaningful improvements over ensemble-based methods in the context of betting market prediction. It is also used to assess the predictive contribution of individual features through feature importance analysis.

3.4 CNN-LSTM Hybrid Network

The CNN–LSTM hybrid architecture was designed to capture both local temporal patterns in the time series and longer-term dependencies between sequences. The model takes input of shape $x \in \mathbb{R}^{T \times F}$ where T denotes the number of time steps ($T = 5$ is the model in this paper), and F is the number of input features (e.g., odds, volume, VWAP transformations, log returns).

3.4.1 Convolutional Feature Extraction

The first stage applies a one-dimensional (1D) convolution across the temporal dimension. With the 64 filters, the activation step output combines and becomes $H \in \mathbb{R}^{T \times 64}$. This layer captures local temporal shifts such as sudden jumps in odds or volume spikes across multiple parallel filters.

Then a MaxPooling layer with pool size 2 is applied. $H_t = \max\{H_{2t}, H_{2t+1}\}$. This halves the temporal dimension, reduces overfitting, and introduces local invariance to small fluctuations. Combined with a spatial dropout layer to randomly set 20% of the entire feature channel to zero to improve generalizing power.

A second convolutional layer with 32 filters is applied to extract higher-level patterns from the pooled representation, yielding $H' \in \mathbb{R}^{T \times 64}$.

3.4.2 Sequential Modeling with LSTMs

The processed features are fed into a stacked LSTM architecture designed to capture evolving trends. The first LSTM layer operates as a sequence-preserving temporal scanner, maintaining 64 hidden units across all time steps to detect and track short-term patterns in the data. This

layer identifies critical dynamics such as momentum shifts in odds movements or increasing temporary imbalances between back and lay volumes. By preserving the complete sequence of hidden states, it ensures that no predictive signals are lost during processing. The second LSTM layer then acts as a strategic aggregator, condensing the temporal sequence into a single 32-unit hidden state that shows the most important parts of the market’s evolution. This layer specializes in recognizing higher-order relationships, connecting market signals to their potential outcomes. Together, these stacked LSTMs form a hierarchical temporal model that matches the way betting markets work, where immediate price movements and longer-term trends interact to create predictive signals. The design of this architecture, which starts with detailed sequence analysis and ends with a compressed summary, is particularly effective for the detection of information that is helpful to the prediction tasks.

3.4.3 Dense Layers and Nonlinear Mapping

The final hidden state $h^{LSTM} \in \mathbb{R}^{32}$ produced by the second LSTM is passed into a series of fully connected layers to enable nonlinear decision boundaries. First, a dense layer with 50 ReLU-activated units expands the feature space to allow more expressive transformations before classification, where the choice of 50 units reflects a trade-off between model capacity and overfitting risk. To improve generalization, a dropout layer randomly sets 30% of the activations at zero during training, thus preventing neuron coadaptation. The output is then passed to a smaller dense layer with 25 ReLU units, which acts as a bottleneck to refine intermediate representations and enforce a more compact encoding. Finally, a single sigmoid-activated neuron produces the prediction, which represents the probability of the positive class, such as whether a horse will lose, making the output interpretable as a confidence score suitable for threshold-based decision-making.

3.4.4 Training Objective and Threshold Calibration

The model is trained using the Adam optimizer with a binary cross-entropy loss function (i.e. $y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})$ where $y \in 0, 1$), and class imbalance is addressed by computing class weights to ensure fair treatment of minority classes. Early stopping with patience of 15 epochs and learning rate reduction on plateau are applied to prevent overfitting and refine convergence. The model is trained and validated on preprocessed sequential data, and predictions are generated for all data splits.

Since in betting applications, precision (positive predictive value) is often more important than overall accuracy (to avoid costly false positives), the final probability output $\hat{y}$ is thresholded at a tuned value t that is not necessarily 0.5. An optimal decision threshold is determined using validation probabilities to maximize precision while maintaining a minimum recall, and this threshold is applied to produce test predictions. The model is then evaluated on classification metrics such as precision, recall, F1-score, and also on simulated profitability from applying

predictions to hypothetical betting strategies.

3.5 XGBoost Implementation

The second predictive framework employed in this study is Extreme Gradient Boosting (XGBoost), a tree-based ensemble algorithm. The inclusion of XGBoost serves two main purposes. First, it provides a methodological contrast to deep learning, allowing us to examine whether static feature representations can achieve competitive predictive performance relative to temporal sequence learning. Second, XGBoost offers interpretability advantages through feature importance scores which reveal the contribution of predictors such as VWAP, log returns, or weight-of-money (WoM). These insights are particularly valuable for financial and betting applications, where model transparency can guide both trading strategies and domain understanding.

The model was trained on engineered tabular features derived from selected variables and optimized using a validation set. A grid search was conducted over a small hyperparameter space, varying the maximum tree depth (3, 4, 5, 6, 7) and the learning rate (0.01, 0.05, 0.1), with the area under the precision recall curve (PR-AUC) in the validation set used as the selection criterion. Class imbalance was addressed through the weighted loss function, calculated as the ratio of negative to positive samples, while early stopping was employed to prevent overfitting by halting training if validation performance failed to improve for 20 rounds. The final model was selected based on its highest PR-AUC score and then used to generate class probabilities for the training, validation, and test sets. An optimal classification threshold was determined from the validation set to maximize precision while satisfying a minimum recall requirement, and the test predictions were derived accordingly.

3.5.1 Baseline

XGBoost is employed as a baseline model for performance comparison across all tasks. For Task 1 and Task 2, the model uses only the most recent log returns of the odds as input features, while for Task 3 it leverages the implied, race-normalized probabilities of winning and losing derived from market odds. This design provides a simple yet informative benchmark, allowing us to evaluate the added value of more complex models and richer temporal feature sets.

3.5.2 Meta Model

To leverage the complementary strengths of both architectures, XGBoost is also employed as a meta-learner to aggregate the outputs of the CNN-LSTM and XGBoost base models. Since the predictions of these models are not entirely overlapping and appear to capture different patterns in the data, stacking them enables the meta model to exploit their distinct decision-making

logic. This ensemble approach is designed to combine the sequential learning capability of CNN-LSTM with the feature-driven interpretability of XGBoost, ultimately aiming to deliver improved predictive performance.

3.6 Model Training Protocol

Both the CNN-LSTM hybrid network and the XGBoost ensemble were trained under a unified training protocol to ensure methodological consistency, fair comparability, and reproducibility of results. This eliminates confounding factors from preprocessing, dataset splits, or evaluation practices, allowing performance differences to be attributed primarily to the model architectures.

For both models, numerical features such as VWAP, odds, and traded volumes were standardized prior to fitting, with min-max normalization applied per race and per feature. This normalization strategy ensures across-race comparability, preventing distortions that would arise from global standardization, as the range of odds can vary substantially between races. By normalizing in this way, values at the same time step across different horses remain on a comparable scale, preventing the model from focusing on absolute values and instead learning the dynamics such as slopes, relative deviations, and short-term trends that are more predictive of outcomes. To avoid data leakage, the scaling parameters were fit exclusively on the training set and subsequently applied to the validation and test sets.

For the CNN-LSTM, an additional step of time-step normalization per feature was applied, centering and scaling each feature sequence within a horse’s timeline. This ensures that the model focuses on relative fluctuations that is whether odds are drifting up or down, or whether traded volume is spiking rather than absolute magnitudes. Without this adjustment, features with higher absolute ranges could dominate the learned representations, leading the CNN-LSTM to overfit to specific race conditions rather than capturing generalizable temporal patterns.

Both models were trained, validated, and tested on identical dataset partitions. The validation set was used exclusively for hyperparameter tuning and early stopping, while the test set was reserved strictly for final evaluation, ensuring unbiased performance assessment.

3.7 Evaluation Framework

Evaluating predictive models in the context of betting exchanges requires metrics that balance statistical accuracy with practical trading relevance. Standard machine learning measures such as accuracy, precision, recall, and F1-score provide an initial assessment of predictive capability, but these metrics alone do not capture whether predictive success translates into financial gain. A model may, for instance, achieve high accuracy by focusing on majority outcomes, yet fail to detect the rarer but more profitable opportunities that drive real trading returns.

To address this gap, a profitability simulation framework was developed, translating model predictions into hypothetical betting strategies executed on the Betfair Exchange under realistic constraints such as commission rates and the timing of position entry and exit. For the odds movement and arbitrage prediction tasks, the strategy operated directionally. When the model detected a profitable back–lay sequence, a back bet was placed at the current odds and later closed with a lay bet once the subsequent odds satisfied the profit condition (3.5). Conversely, when a lay–back sequence was predicted, the trade was initiated with a lay bet at the current odds and later closed with a back bet under the same profitability criteria.

For race outcome prediction, the evaluation framework simulated betting strategies by backing or laying horses based on the model’s forecasts of winning or losing outcomes. When a backed horse won, the position generated a profit of $0.95 \times (b - 1)$, net of the exchange’s 5% commission, where b denotes the back odds. Conversely, a lay position on a winning horse incurred a liability of $l - 1$, with l representing the lay odds. If the horse lost, the back position incurred a fixed liability of 1, while the lay position yielded a profit of 0.95. This evaluation methodology allowed model performance to be assessed in terms of real-world financial returns, providing a more practical measure of predictive utility than statistical accuracy metrics alone.

Betting performance was evaluated using total return, average return per bet, and confidence interval bounds, capturing not only profitability but also the variability and robustness of outcomes. By combining statistical accuracy with profitability-based evaluation, the framework provides a holistic assessment of model effectiveness, ensuring that predictive success is judged both in terms of machine learning performance and practical viability in live betting markets.

4 Result and Discussion

4.1 Descriptive Statistics of the Dataset

The predictive frameworks of CNN–LSTM and XGBoost were evaluated on a dataset comprising 1701 horse races conducted on the Betfair Exchange platform. Each race included an average of 8 participating horses, with time series data spanning 10 minutes prior to race commencement. Features engineered from raw odds and volume streams included VWAP, weight of money (WoM), log returns.

Table 4.1 presents the summary statistics of features computed over historical windows, where each statistic is based on the mean value of the feature within its respective time window. Odds-related variables, such as back and lay prices, exhibit strong right-skewness: median prices are 12.39 and 12.95, respectively, while maximum values exceed 700 and 900, reflecting the presence of long-shot horses with very high odds. Traded volumes show similar skewness, with medians of 37.11 (back) and 23.81 (lay), but extreme values surpass 20,000 for back volumes. Volume-weighted average prices (VWAP) follow similar patterns to raw prices, whereas price–volume

Table 4.1: Summary statistics of features

Feature	min	Q1	Q2	Q3	max	mean	mean 95% CI	
							Lower	Upper
Raw Data								
back prices	1.084	6.170	12.390	29.900	776	37.245	36.009	38.481
back volumes	0.284	15.232	37.111	81.780	21825.365	79.163	73.850	84.477
lay prices	1.094	6.350	12.950	31.800	935	41.688	40.209	43.166
lay volumes	0.207	9.917	23.811	55.722	15769.614	58.532	54.244	62.820
back VWAP	1.078	6.075	12.102	29.214	765.811	36.145	34.944	37.347
lay VWAP	1.099	6.456	13.231	32.673	940.540	43.063	41.531	44.594
back PVT	-781.287	-4.474	0.369	5.320	683.984	0.338	-0.007	0.683
lay PVT	-638.695	-3.350	0.120	3.431	422.924	-0.567	-0.861	-0.272
WoM	0.286	0.558	0.603	0.647	0.884	0.602	0.601	0.604
back log return	-0.319	-0.014	0.002	0.021	0.216	0.004	0.004	0.005
lay log return	-0.343	-0.015	0.002	0.021	0.246	0.004	0.003	0.004
Scaled Data								
back prices	0	0.024	0.104	0.349	1	0.242	0.240	0.245
back volumes	0	0.043	0.116	0.279	0.918	0.183	0.181	0.184
lay prices	0	0.022	0.098	0.339	1	0.236	0.234	0.239
lay volumes	0	0.034	0.093	0.240	0.963	0.164	0.163	0.165
back VWAP	0	0.025	0.105	0.351	0.999	0.243	0.241	0.245
lay VWAP	0	0.022	0.096	0.338	0.998	0.236	0.233	0.238

trends (PVT) are centered near zero and display less skewness. The weight of money (WoM), which captures order book imbalances, averages around 0.60, suggesting a slight bias toward back bets over lay bets, however, log returns for both back and lay prices are tightly distributed around zero, indicating no persistent directional trend in price movements.

To address variation in ranges between races in each features, all numerical variables were normalized using min-max scaling applied per race and per feature. As summarized in Table 4.1, this transformation constrains all features to the $[0, 1]$ range, ensuring comparability of values across horses and races, and allowing the models to focus on relative dynamics rather than absolute scales.

4.2 Model Performance on Classification Tasks

The evaluation results across the three prediction tasks emphasize a deliberate focus on precision as the primary performance metric, aligning with the real-world objective of minimizing losses

Table 4.2: Model Performance Comparison for Prediction Tasks

Model	Target	F1-Score	Precision	Recall	Accuracy	Log Loss	Log Loss CI	
							Lower	Upper
Task 1								
CNN-LSTM	Back Odds	0.0267	0.6747	0.0136	0.4944	0.6915	0.6909	0.6921
	Lay Odds	0.0446	0.6569	0.0231	0.4950	0.6919	0.6911	0.6927
XGBoost	Back Odds	0.0618	0.5998	0.0326	0.4963	0.6923	0.6921	0.6926
	Lay Odds	0.0230	0.6182	0.0117	0.4917	0.6926	0.6917	0.6935
Baseline	Back Odds	0.0441	0.6197	0.0229	0.4953	0.6922	0.6917	0.6926
	Lay Odds	0.0853	0.5498	0.0462	0.4937	0.6924	0.6920	0.6928
Meta	Back Odds	0.0241	0.6494	0.0123	0.4937	0.6914	0.6907	0.6922
	Lay Odds	0.0368	0.6616	0.0189	0.4941	0.6920	0.6909	0.6930
Task 2								
CNN-LSTM	Back-Lay Seq	0.0720	0.4920	0.0389	0.5830	0.6981	0.6970	0.6992
	Lay-Back Seq	0.0914	0.5266	0.0500	0.5440	0.6902	0.6890	0.6914
XGBoost	Back-Lay Seq	0.0155	0.4577	0.0079	0.5829	0.6914	0.6910	0.6918
	Lay-Back Seq	0.0191	0.5590	0.0097	0.5426	0.6887	0.6871	0.6902
Baseline	Back-Lay Seq	0.0211	0.4429	0.0108	0.5824	0.6884	0.6875	0.6893
	Lay-Back Seq	0.0205	0.4318	0.0105	0.5401	0.6921	0.6919	0.6923
Meta	Back-Lay Seq	0.0261	0.5274	0.0134	0.5841	0.6748	0.6729	0.6768
	Lay-Back Seq	0.1366	0.5603	0.0778	0.5493	0.6833	0.6817	0.6850
Task 3								
CNN-LSTM	Win	0.1329	0.5453	0.0757	0.8872	0.5457	0.5411	0.5503
	Lose	0.0535	0.9811	0.0275	0.1381	0.5379	0.5331	0.5429
XGBoost	Win	0.0181	0.6230	0.0092	0.8862	0.5200	0.5165	0.5236
	Lose	0.1458	0.9875	0.0787	0.1831	0.5141	0.5101	0.5182
Baseline	Win	0.0167	0.6364	0.0085	0.8862	0.5371	0.5330	0.5415
	Lose	0.0760	1.0000	0.0395	0.1492	0.5467	0.5429	0.5509
Meta	Win	0.0601	0.5874	0.0317	0.8869	0.3084	0.3020	0.3150
	Lose	0.3862	0.9800	0.2405	0.3229	0.3081	0.3019	0.3145

in betting scenarios. Decision thresholds for all models were explicitly optimized to maximize precision, resulting in relatively low F1-scores. This trade-off is intentional, by prioritizing high precision, the models produce fewer positive predictions but with higher confidence, which is

critical when predictions inform financial decisions such as backing or laying horses.

For feature design, Tasks 1 (short-term odds movement) use VWAP, WoM, and log returns while task 2 (arbitrage detection) use betted volume, VWAP, PVT, and WoM. Task 3 (race outcome prediction) additionally incorporates implied probabilities derived from odds.

In Task 1, class distributions are relatively balanced, with approximately 51% of odds increasing within the next 60 seconds. Similarly, in Task 2, around 43% of time points present a risk-free arbitrage opportunity, indicating minimal class imbalance in both tasks. Despite this, all models exhibit low recall, a direct consequence of optimizing for precision rather than balanced classification performance.

The CNN-LSTM model consistently achieves the highest precision across both tasks. In Task 1, it correctly identifies upward price movements with a precision of over 65%, while both XGBoost and the baseline remain below 62%. For Task 2, CNN-LSTM maintains a precision of around 50%, significantly higher than the 43% precision from the baseline model, which is close to random guessing given the class proportions. XGBoost demonstrates relatively strong performance in detecting lay-back arbitrage opportunities, achieving a precision of 55%, though its substantially lower recall prevents it from outperforming CNN-LSTM overall.

Task 3 presents a highly imbalanced dataset, as only one horse wins per race. The baseline model, using market-implied probabilities, outperforms both CNN-LSTM and XGBoost in predicting both winning and losing horses. This suggests that market odds are already an effective aggregator of available information, and additional features introduce noise, leading to overfitting. Interestingly, subsequent profit simulations reveal that CNN-LSTM displays unexpected behaviors that merit further investigation, despite its weaker precision metrics in this task.

The meta model, built by stacking CNN-LSTM and XGBoost outputs, shows no significant improvement in Tasks 1 and 3 but achieves notable gains in Task 2, raising precision to approximately 54%, a meaningful improvement over the individual models.

4.3 Profitability Simulation Results

Table 4.3 summarizes the results of profitability simulations across all three prediction tasks, reporting both total and per-bet average returns with 95% confidence intervals, along with the proportion of profitable bets (Positive Rate) and the number of bets executed under each strategy. These results illustrate how precision-optimized models translate their predictive performance into financial outcomes.

In Task 1, which focuses on short-term odds movement prediction, none of the models achieved profitability. CNN-LSTM performed comparatively better than XGBoost and the baseline, with average returns of -0.00050 per bet for back odds and -0.00094 for lay odds, while XG-

Table 4.3: Model Performance Comparison for All Prediction Tasks

Model	Target	Total	Average	Avg. 95% CI		Positive Rate	# Bets
				Lower	Upper		
Task 1							
CNN-LSTM	Back Odds	-0.18	-0.00050	-0.00107	0.00008	0.511	372
	Lay Odds	-0.61	-0.00094	-0.00158	-0.00030	0.503	650
XGBoost	Back Odds	-2.72	-0.00272	-0.00345	-0.00199	0.487	1002
	Lay Odds	-0.68	-0.00194	-0.00281	-0.00108	0.481	351
Baseline	Back Odds	-1.86	-0.00273	-0.00353	-0.00193	0.492	681
	Lay Odds	-6.47	-0.00416	-0.00491	-0.00341	0.464	1555
Meta	Back Odds	-0.68	-0.00195	-0.00279	-0.00110	0.463	348
	Lay Odds	-0.81	-0.00152	-0.00229	-0.00076	0.514	529
Task 2							
CNN-LSTM	Back-Lay Seq	-6.76	-0.00567	-0.00660	-0.00475	0.492	1191
	Lay-Back Seq	-11.17	-0.00709	-0.00822	-0.00596	0.527	1576
XGBoost	Back-Lay Seq	-2.05	-0.00790	-0.01084	-0.00496	0.458	260
	Lay-Back Seq	-1.17	-0.00407	-0.00672	-0.00143	0.559	288
Baseline	Back-Lay Seq	-2.56	-0.00696	-0.00894	-0.00498	0.443	368
	Lay-Back Seq	-1.55	-0.00384	-0.00513	-0.00254	0.432	403
Meta	Back-Lay Seq	-3.30	-0.00860	-0.01130	-0.00590	0.527	383
	Lay-Back Seq	-12.74	-0.00553	-0.00649	-0.00458	0.560	2304
Task 3							
CNN-LSTM	Win	16.29	0.02837	-0.14895	0.20569	0.545	574
	Lose	-1193.10	-1.32714	-2.65483	0.00055	0.981	899
XGBoost	Win	-14.56	-0.23875	-0.39229	-0.08522	0.623	61
	Lose	-1970.25	-0.77053	-1.43661	-0.10446	0.987	2557
Baseline	Win	-12.29	-0.22338	-0.38402	-0.06274	0.636	55
	Lose	1203.65	0.95000	0.95000	0.95000	1.000	1267
Meta	Win	-34.01	-0.15252	-0.24939	-0.05565	0.587	223
	Lose	-2104.10	-0.26739	-0.53267	-0.00211	0.980	7869

Note: Total observations (N) = 36,205 for all models. Confidence intervals are for the average return per bet.

Positive Rate is the rate of bets yielding positive profit.

For task 1 and 2, the value at risk is 1 per bet.

Boost recorded -0.00272 and -0.00194 respectively. The baseline strategy performed the worst, particularly in lay bets, losing a total of -6.47 across 1,555 bets. The meta-model offered no meaningful improvement. These results highlight that, although the models were explicitly calibrated for high precision, the profits from correct predictions were still insufficient to offset losses. This suggests that odds movement forecasts may hold more value as a complementary decision-support tool, helping bettors to time wagers more effectively, rather than serving as a profitable strategy on their own.

Task 2, which focuses on detecting arbitrage opportunities, showed modestly improved stability in predictions but again failed to deliver consistent profits. CNN-LSTM generated the largest number of trades, placing more than 2,700 bets combined, but incurred notable losses, particularly in lay-back sequences, where the average return per bet was -0.00709 . XGBoost performed somewhat better, especially in lay-back sequences, achieving a Positive Rate of 0.559 while limiting losses to -0.00407 per bet, the smallest average loss in this category. The meta-model increased precision slightly and reduced the average loss but also triggered a much higher number of bets, leading to large total losses. These findings indicate that while arbitrage signals are more robust than short-term movement predictions and exhibit higher-than-random accuracy, the profit from positive trades are insufficient to compensate the losses, positioning these models as potentially valuable decision-support tools rather than fully automated trading systems.

Task 3 reveals a clear contrast between backing and laying strategies. Although the CNN-LSTM model achieves slightly lower precision than the baseline, it manages to produce a modest positive total return of 16.29 on win bets. This suggests that CNN-LSTM is able to extract valuable information, such as sudden spikes in odds that may reflect hidden market information, from temporal patterns in price and volume data. This enables the model to identify profitable opportunities on horses with higher odds. However, the lower CI of the average profit per bet of CNN-LSTM is negative, indicating that this may not be a stable way of profiting.

In contrast, lay betting proves considerably more challenging. While all models achieve high positive rates when predicting losing horses, the inherent risk structure of lay bets means that occasional liabilities can be very large, particularly when betting against horses with high odds. CNN-LSTM appears less effective at leveraging time-series patterns for loser prediction, as temporal features provide limited incremental value beyond the strong predictive power of market-implied probabilities. This maybe because the hidden pattern in market odds only exists in certain winning horses and it is not universal, therefore CNN-LSTM was not able to avoid picking winning horses when the task was identifying losing ones. This also means that the actual performance of CNN-LSTM in predicting winner needs to be further investigated to see if the profit made was simply coincident.

The baseline model, which relies solely on these probabilities, delivers exceptional performance in lay bets, achieving a total return of 1,203.65 with a perfect positive rate. However, the practical profitability of this strategy remains uncertain, as a single large liability could offset

accumulated gains, highlighting the need for more extensive data to validate its robustness. Both XGBoost and the meta-model underperform the baseline, incurring substantial total losses due to overbetting and reduced precision.

Overall, these findings underscore that precision-oriented calibration limits risk exposure and is especially effective in highly imbalanced tasks such as race outcome prediction. While odds movement and arbitrage models remain unprofitable under the current framework, their signals may complement broader strategies by informing timing and opportunity selection. In practice, these results suggest that rather than attempting to predict winners outright, machine learning models may be more effectively deployed to filter out losing horses and refine betting selections.

4.4 Saliency Map

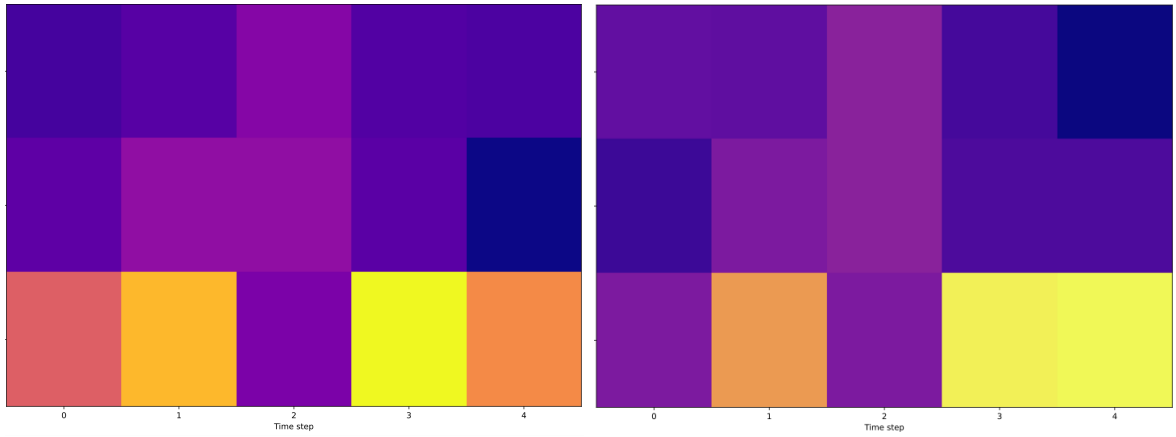


Figure 1: Task 1 (left: Back Odds, right: Lay Odds)

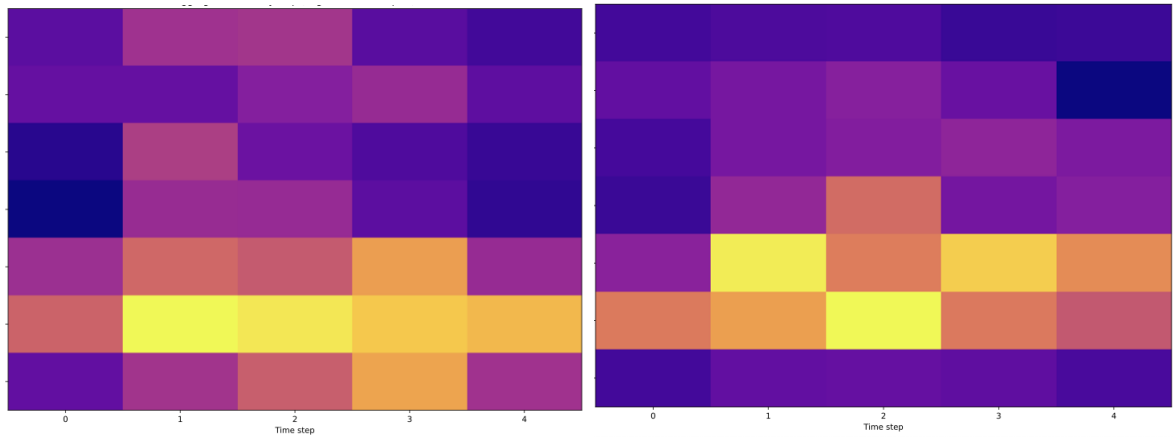


Figure 2: Task 2 (left: Back-Lay Seq, right: Lay-Back Seq)

To better understand the decision-making process of the CNN-LSTM model, saliency maps were generated for the three prediction tasks. Saliency maps are a widely used explainability tool in deep learning that highlight the input regions most influential for a given prediction. In this context, the heatmaps indicate which time steps and features (Table 4.4 has the features name of each task) contribute most strongly to the model's output.

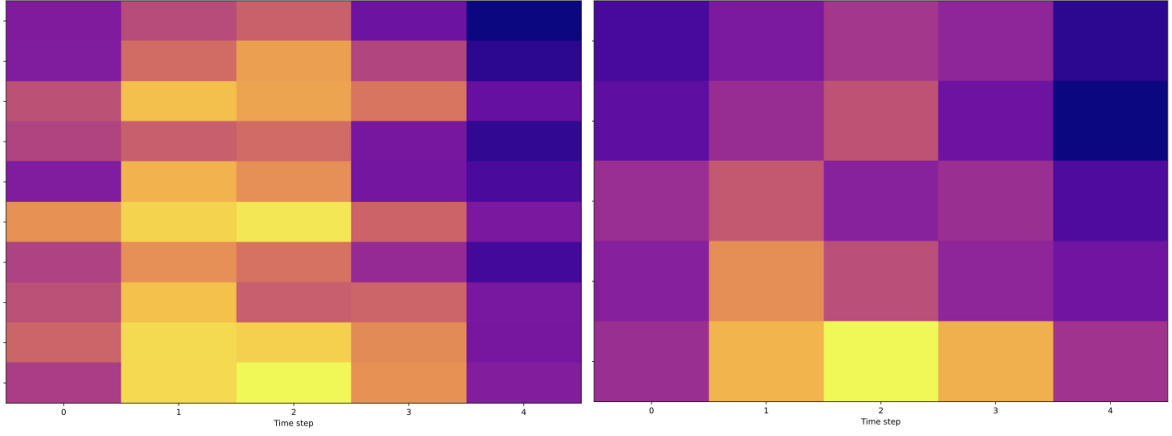


Figure 3: Task 3 (left: Win, right: Lose)

Mathematically, saliency maps are derived by computing the gradient of the model’s output with respect to the input features. The magnitude of this gradient reflects how sensitive the model’s prediction is to changes in the input, with larger values indicating higher importance. This approach provides a first-order approximation of feature relevance, helping to visualize the model’s focus without requiring explicit interpretability constraints.

Figures 1, 2, and 3 present the saliency maps for each prediction task. For Task 1, the model shows localized sensitivity to sudden changes in odds log returns, with stronger gradients at later time steps, indicating that recent price movements heavily influence predictions. Task 2 exhibits a broader distribution of importance across time steps, suggesting that detecting arbitrage opportunities is more complex. Interestingly, the model appears particularly influenced by opponents’ PVT values, highlighting that overall odds trends, rather than single price points, are key to identifying risk-free betting opportunities. In Task 3, saliency maps show strong activations across multiple time steps when predicting winners, implying that the CNN-LSTM model focuses on temporal market dynamics rather than static snapshots. However, the heatmaps also display noisy gradients, which aligns with the model’s limited precision and its inability to consistently outperform market-implied probabilities. For predicting losing horses, implied losing probability dominates feature importance, while other features contribute minimally.

A notable pattern in both Task 2 and Task 3 saliency maps is the concentration of importance around time steps approximately two minutes prior to the prediction point. This suggests that the CNN-LSTM model is not simply relying on absolute odds values but is instead detecting meaningful patterns in temporal sequences.

Overall, these visualizations demonstrate that the CNN-LSTM model effectively captures sequential betting market dynamics, particularly for short-term price movement predictions (Tasks 1 and 2). The high variability in Task 3 saliency maps underscores the inherent difficulty of directly predicting race outcomes from odds data.

4.5 Feature Importance

Table 4.4: Feature Importance

Feature	Importance (%)	Feature	Importance (%)
Task 1: Back Odds		Task 1: Lay Odds	
Back VWAP	19.7	Lay VWAP	23.3
WoM	18.5	WoM	26.9
Back Log Return	61.8	Lay Log Return	49.8
Task 2: Back-Lay Seq		Task 2: Lay-Back Seq	
Back Betted Volumes	22.0	Back Betted Volumes	12.0
Lay Betted Volumes	9.6	Lay Betted Volumes	18.9
Back VWAP	17.8	Back VWAP	24.6
Lay VWAP	10.9	Lay VWAP	7.3
Opponents' Back PVT	3.0	Opponents' Back PVT	8.2
Opponents' Lay PVT	19.5	Opponents' Lay PVT	10.0
WoM	17.3	WoM	18.9
Task 3: Win		Task 3: Lose	
Back VWAP	7.4	Lay Betted Volumes	31.1
Lay VWAP	19.7	Lay VWAP	25.8
Opponents' Back PVT	8.1	WoM	17.6
Back PVT	23.7	Lay Log Return	20.1
Lay PVT	4.5	Lose Prob	5.5
WoM	10.9		
Back Log Return	2.7		
Lay Log Return	17.5		
Win Prob	3.6		
Lose Prob	1.9		

Feature selection was performed through forward and backward subset selection based on validation performance. Then, the feature importance was estimated using SHAP values from XGBoost models trained on the same feature sets as CNN-LSTM. Since CNN-LSTM is difficult to interpret directly, XGBoost with SHAP provides insight into which features drive predictions.

Table 4.4 shows that odds movement prediction (Task 1) is dominated by price momentum features such as log returns, confirming that short-term price trends are key drivers of odds shifts. Arbitrage detection (Task 2) relies on a more balanced mix of price and liquidity signals, including VWAP, WoM, and traded volumes, highlighting the importance of both price efficiency and order book imbalances in identifying mispricings.

Race outcome prediction (Task 3) shows a stronger emphasis on lay-side activity, particularly lay bet volumes, VWAP, and log returns, which dominate importance for loser predictions. Interestingly, winner prediction distributes importance more evenly across features, suggesting that detecting under-priced winners is more challenging and may require subtle temporal patterns, which CNN-LSTM is better suited to capture. The strong reliance on lay-side signals indicates that bettors collectively provide valuable information about likely losers, whereas identifying potential winners involves less explicit signals.

However, many features (e.g., log returns, VWAP, WoM) are highly correlated, so SHAP importance can be split across related variables. Grouping features into broader categories may provide more robust interpretation.

4.6 Comparison Between CNN-LSTM and XGBoost

Experimental results highlight distinct strengths and weaknesses of the CNN-LSTM and XGBoost models across the three prediction tasks. In Task 1, CNN-LSTM outperforms XGBoost by effectively modeling high-frequency fluctuations in market prices, leveraging its sequential modeling capabilities. Similarly, in Task 2, CNN-LSTM demonstrates better predictive power due to its ability to capture temporal dependencies in odds sequences. In Task 3 (race outcome prediction), however, both models struggle to outperform market-implied probabilities, underscoring the efficiency of betting exchanges and the difficulty of extracting additional predictive signals.

Although CNN-LSTM shows some potential to identify profitable betting opportunities by focusing on temporal patterns rather than absolute odds, its overall precision remains limited. XGBoost, while robust and interpretable, similarly fails to consistently generate profit. This further emphasizes the challenge of surpassing aggregate market beliefs, which already incorporate a wide range of public and private information.

An analysis of the saliency maps (for CNN-LSTM) and SHAP-based feature importance (for XGBoost) provides insight into these performance differences. For **Task 2**, CNN-LSTM focuses heavily on opponents' PVT (price-volume trend), while XGBoost emphasizes VWAP (volume-weighted average price) and betting volumes. Their complementary focus explains why the meta-model, which combines predictions from both models, achieves a significant performance improvement for this task. In contrast, for **Task 1**, both models rely primarily on odds log returns, limiting the potential benefit of model stacking. For **Task 3**, feature importance is distributed relatively evenly across variables in both models, reducing the effectiveness of a simple decision tree meta-model in aggregating their strengths.

Overall, CNN-LSTM excels at capturing sequential dynamics, while XGBoost offers interpretability and robustness in modeling tabular features. Their combination yields notable improvements only when the models exploit distinct patterns, as demonstrated in Task 2, high-

lighting the importance of complementary modeling approaches in hybrid prediction frameworks.

4.7 Limitations

While the findings provide valuable insights, several limitations should be acknowledged. The models were trained and evaluated on a single betting exchange dataset, where odds and volumes may not reflect the long term performance correctly. Moreover, both models were explicitly calibrated to prioritize precision over recall, reflecting the business requirement to minimize costly false positives in betting applications. Although this calibration is appropriate for risk-averse contexts, it inevitably reduces the number of betting opportunities and constrains potential profitability.

The profitability simulations are subject to several simplifying assumptions. Bets were modeled as being of constant size, executed without transaction costs, and always placed successfully. Although VWAP was employed to approximate execution prices and mitigate the effect of partial filling in real markets, the setup remains an idealized environment. In reality, frictions such as liquidity constraints, and failed order execution could substantially erode profitability, particularly for arbitrage-based strategies that are highly sensitive to timing and execution speed. Moreover, the temporal scope of the dataset poses additional challenges. Odds movements are inherently non-stationary and may shift as market behavior evolves, meaning that models performing well historically could degrade quickly in live trading without continual retraining and recalibration. Computational considerations also present practical limitations. CNN-LSTM, while effective in capturing sequential patterns, requires significantly greater computational resources for both training and deployment compared to XGBoost. This restricts its feasibility in environments where low latency and high scalability are critical to execution.

4.8 Practical Implications for Business Application

From a business perspective, these findings suggest that machine learning models can play a valuable role in betting strategy design, but they are not yet ready to operate as fully automated profit generating systems. The CNN-LSTM model is best suited as a screening tool to highlight horses with a high likelihood of winning, helping traders or automated systems focus attention on the most promising opportunities. Conversely, XGBoost demonstrates stronger performance in identifying likely losers, offering a complementary perspective. While not every signal will lead directly to profit, the combination of these models increases coverage and reduces the chance of overlooking potentially profitable bets.

Notably, the classification outputs from the two models are not fully aligned, reflecting that they capture patterns in the data using different logic and reasoning. CNN-LSTM emphasizes sequential dependencies and temporal dynamics, while XGBoost leverages feature interactions

and decision tree-based reasoning. This complementarity suggests there is potential for hybrid approaches. Although a simple decision-tree meta-model was tested, it is insufficient to effectively combine their strengths, highlighting the need for more well-suited ensembling techniques to balance CNN-LSTM and XGBoost's strengths.

The lack of consistent profitability in odds movement prediction (Task 1) and arbitrage detection (Task 2) further indicates that these models are most effective as decision-support tools rather than standalone betting engines. They could enhance professional bettors' decision-making, provide real-time alerts, or act as a secondary input alongside traditional statistical approaches. Beyond betting, these insights are broadly relevant to other industries facing sequential prediction challenges under uncertainty, such as financial trading, logistics optimization, and risk management.

5 Conclusion

This research explored the use of advanced machine learning techniques, specifically a CNN-LSTM hybrid architecture and XGBoost, for predicting both odds movements and race outcomes in horse racing betting exchanges. By treating betting markets as complex time-series environments analogous to financial exchanges, the study highlights the importance of temporal pattern in predicting behavior of horse racing market. CNN-LSTM networks proved suitable for extracting local patterns and long-term temporal dependencies in price and volume trajectories, while XGBoost offered a complementary strength in modeling nonlinear interactions between market indicators, and engineered features.

The literature review demonstrated that traditional statistical and econometric methods, such as ARIMA and logistic regression, struggle to account for the nonlinear and rapidly evolving nature of betting exchanges. Although these models provide interpretability and a solid theoretical foundation, they fail to exploit the richness of high-frequency data. Deep learning methods, by contrast, provide powerful tools for uncovering complex temporal patterns, though often at the cost of interpretability. Ensemble methods like XGBoost address this trade-off by combining high predictive performance with insights into feature importance, enabling a more thorough understanding of market dynamics.

The results demonstrate that the CNN-LSTM model consistently outperformed XGBoost in both the odds movement and arbitrage detection tasks. The ability of CNN-LSTM to capture temporal pattern in betting data proved advantageous over the static feature of market characteristics in tree-based approach of XGBoost. Furthermore, when the two models were combined, the resulting hybrid system achieved superior performance in the arbitrage detection task, highlighting the complementary strengths of deep sequence modeling and gradient-boosted decision trees.

In the outcome prediction task, CNN-LSTM exhibited an ability to uncover subtle trading

patterns and price behaviors rather than relying on absolute price that suggested winning horses. However, despite this promising insight, the average profit per bet of the CNN-LSTM model remained low, making it challenging to achieve stable profitability. Moreover, both CNN-LSTM and XGBoost fell short of outperforming the market-implied probabilities embedded in exchange odds when predicting losing horses, indicating that market odds are indeed powerful indicators of the race outcome.

These findings emphasize both the potential and the limitations of applying machine learning to betting market prediction. While deep learning architectures like CNN-LSTM are effective in identifying complex temporal patterns and enhancing prediction accuracy, achieving a consistent edge over market prices remains difficult. Future work may focus on refining feature engineering, integrating richer contextual information such as weather conditions and horse health metrics, and exploring meta model to aggregate predictive power of different models. By advancing these methods, predictive modeling in sports betting could evolve beyond decision-support systems, and develop automated betting models.

References

- Ali, M. M. (1998). Probability models on horse-race outcomes. *Journal of Applied Statistics*.
- Alqaraawi, A., M. Schuessler, P. Weiß, E. Costanza, and N. Berthouze (2020). Evaluating saliency map explanations for convolutional neural networks: A user study.
- Bacon-Shone, J., V. S. Y. Lo, and K. Busche (1992). Logistic analyses for complicated bets. Research report, Department of Statistics, University of Hong Kong, Hong Kong.
- Bunyan, A. (2014). Time series analysis and forecasting of in-play odds on a betting exchange. Master’s thesis, School of Computing, National College of Ireland.
- Chen, T. and C. Guestrin (2016). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- Dzalbs, I. and T. Kalganova (2018). Forecasting price movements in betting exchanges using cartesian genetic programming and ann.
- Gonçalves, R., V. M. Ribeiro, F. L. Pereira, and A. P. Rocha (2019). Deep learning in exchange markets.
- Goodacre, J. and B. Schlagman (2024, July 29). Algorithmic trading in financial and sports (exchanges). *SSRN Electronic Journal*.
- Harville, D. A. (1973, June). Assigning probabilities to the outcomes of multi-entry competitions. *Journal of the American Statistical Association*.
- Ichiishi, T. (1983). *Game Theory for Economic Analysis*. New York.
- Lu, W., J. Li, Y. Li, A. Sun, and J. Wang (2020). A cnn-lstm-based model to forecast stock prices. *Complexity*.
- Lundberg, S. M. and S.-I. Lee (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*.
- Santos, J. (2014). A trading agent framework using plain strategies & machine learning. Master’s thesis, Universidade do Porto.
- Shin, H. S. (1991, September). Optimal betting odds against insider traders. *The Economic Journal*.
- Terawong, C. and D. Cliff (2024, January). Xgboost learning of dynamic wager placement for in-play betting on an agent-based model of a sports betting exchange. *arXiv preprint*.
- Øvregård, (2008). *Trading “in-play” betting Exchange Markets with Artificial Neural Networks*. Ph. D. thesis. Master’s thesis or dissertation.

Appendices

A Code Implementation

The complete implementation can be found at:

<https://github.com/AlanCheung1115/STAT-0034-Code.git>

B Arbitrage Derivation

Consider a betting exchange with commission rate $c \in [0, 1)$. In a single race, the bets are:

- A back bet of amount $x > 0$ at odds $b > 1$
- A lay bet of amount $y > 0$ at odds $l > 1$

B.0.1 Capital Allocation

The total money at risk $M > 0$ must satisfy:

$$x + y(l - 1) = M \implies y = \frac{M - x}{l - 1} \quad (\text{B.1})$$

B.0.2 Profit Conditions

The profit functions for both outcomes after considering the total money at risk and commissions are:

$$\pi_{\text{win}} = (1 - c)(x(b - 1) - y(l - 1)) = (1 - c)(bx - yl - x + y) \quad (\text{B.2})$$

$$\pi_{\text{lose}} = (1 - c)(y - x) \quad (\text{B.3})$$

B.0.3 Arbitrage Solution

Setting $\pi_{\text{win}} = \pi_{\text{lose}}$ yields $bx = yl$ and substitute (B.1) into it:

$$x = \frac{lM}{bl - b + l}, \quad y = \frac{bM}{bl - b + l} \quad (\text{B.4})$$

Substituting (B.4) into (B.3) gives the riskless profit:

$$\pi = \frac{(b - l)(1 - c)M}{bl - b + l} \quad (\text{B.5})$$

B.0.4 Positivity Conditions

The denominator simplifies to:

$$bl - b + l = b(l - 1) + l > 0 \quad \forall b, l > 1 \quad (\text{B.6})$$

Thus, positive riskless profit ($\pi > 0$) occurs when:

$$b > l \quad (\text{B.7})$$

C Use of GenAI

I acknowledge the use of ChatGPT (version GPT-5) to summarize my initial notes and to proofread my final draft.